



Photo by anita_peter on pixabay

Lecture 7

—

Support Vector Machines (SVM)

Aurorafalter (*Anthocharis cardamines*)

- ♦ Der Aurorafalter ist ein Schmetterling (Tagfalter) aus der Familie der Weißlinge. Die Falter erreichen eine Flügelspannweite von 35 bis 45 Millimetern.
- ♦ Die Flügel beider Geschlechter sind weiß, wobei die äußerste Spitze der Vorderflügel grau bis grau-schwarz gefärbt ist und etwa in der Mitte dieser Flügel ein kleiner schwarzer Punkt zu erkennen ist. Die äußere Hälfte der Vorderflügel der Männchen ist auffallend orange gefärbt, diese Färbung fehlt beim Weibchen.
- ♦ Die Aurorafalter sind im gesamten Europa und im Mittleren Osten weit verbreitet. Er war Schmetterling des Jahres 2004. Die Tiere sind Einzelgänger und überstehen den Winter als Puppen, aus denen sie dann im Frühjahr ausschlüpfen.



Weiblicher Aurorafalter (Unterseite) - Photo by Ralphs_Fotos on pixabay



Weiblicher Aurorafalter - Photo by jggrz on pixabay

Quellen:

* <https://de.wikipedia.org/wiki/Aurorafalter>

* <https://www.pantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten>



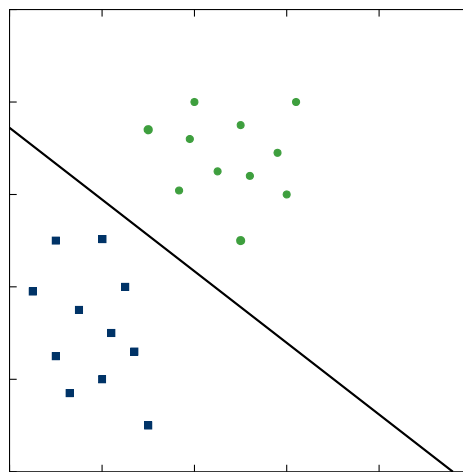
Classification – Support Vector Machines (SVM)

1. Hard Margin SVM
2. Feature Transformations
3. The Kernel Trick
4. Regularization - Soft Margin SVM
5. SVM in practice

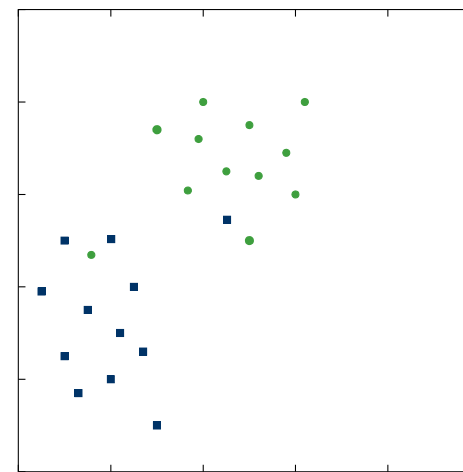


Motivation

- ♦ assume two linearly separable classes
- ♦ compute linear decision boundary that
 - allows for separation of training data
 - generalizes well



linearly separable classes

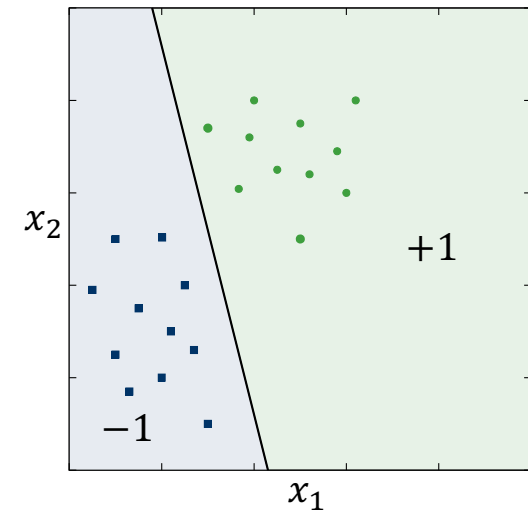
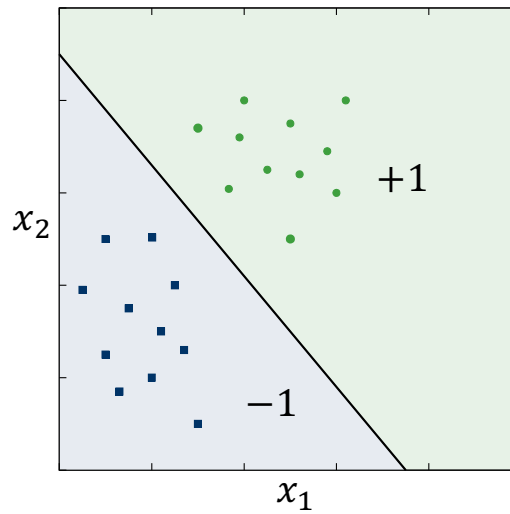
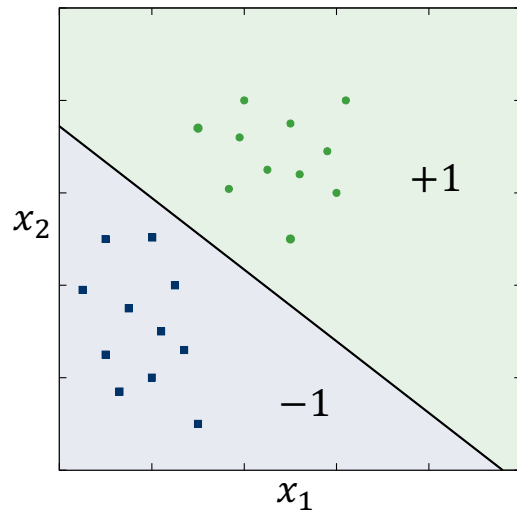


not linearly separable classes



Motivation

- ♦ Many, many solutions...





Optimal Separating Hyperplane

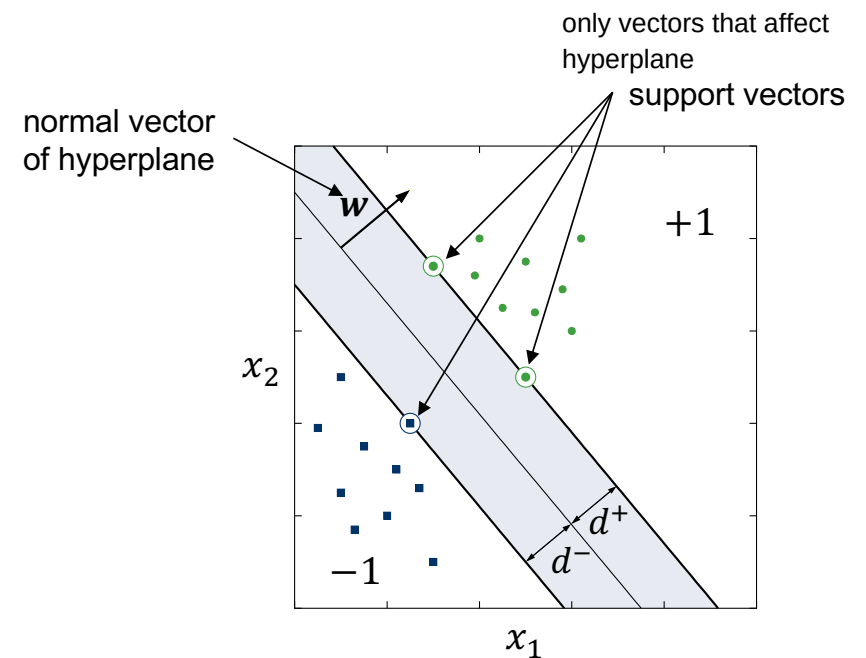
Vapnik 1996: Optimal separating hyperplane that

- separates two classes and
- maximizes the distance to the closest point from either class (called “support vectors”)

has to be at least three vectors but can be more

This results in

- unique solution for hyperplanes, and
- (in most cases) better generalization.

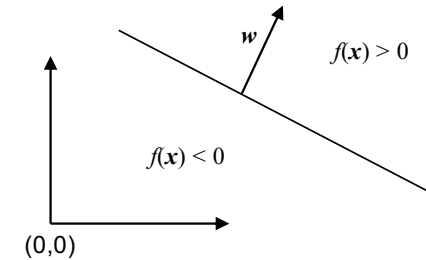


Functional margin of separating hyperplane: $d^+ + d^-$



Plane equation and distance

- ◆ Plane equation: $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$
 - normal vector: $\mathbf{w}$
 - point on plane: $f(\mathbf{x}) = 0$
 - point above plane: $f(\mathbf{x}) > 0$
 - point below plane: $f(\mathbf{x}) < 0$
 - "above" = in direction of plane normal



- ◆ (Signed) distance d of a point $\mathbf{x}$ to the hyperplane: $d = \frac{\mathbf{w}^T \mathbf{x} + w_0}{\|\mathbf{w}\|}$



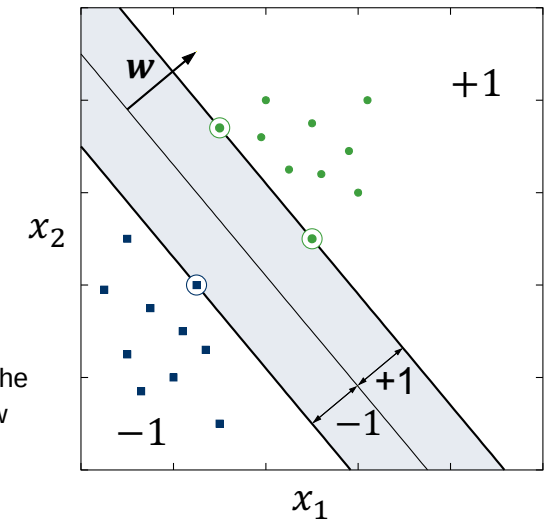
SVM – Classification

◆ Given

- data point: x_i
- class of data point x_i is $y_i \in \{-1, +1\}$

➔ Classifier: $g(x_i) = \text{sgn}(\mathbf{w}^T x_i + w_0)$

w_0 is offset or bias term representing the distance of the hyperplane from the origin along the normal vector $\mathbf{w}$



◆ Functional margin of x_i : $y_i (\mathbf{w}^T x_i + w_0)$ if a measure of the “quality” of the classification

- Problem: functional margin can be increased/decreased by scaling plane equation
- Solution: constrain functional margin to be at least 1 for each (training) point: $y_i (\mathbf{w}^T x_i + w_0) \geq 1$
➔ support vectors have functional margin of +1 (i.e., $(\mathbf{w}^T x_i + w_0)$ is +1 or -1 for each support vector)

◆ Margin of the classifier: $\frac{2}{\|\mathbf{w}\|}$

(twice the minimal distance $d = \frac{\mathbf{w}^T x + w_0}{\|\mathbf{w}\|}$ of all (training) points from the hyperplane)



SVM – Training

♦ Training =

- find hyperplane maximizing the margin $\frac{2}{\|w\|}$
- subject to constraint $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1$ for all data points

This is hard to solve directly! Better formulation:

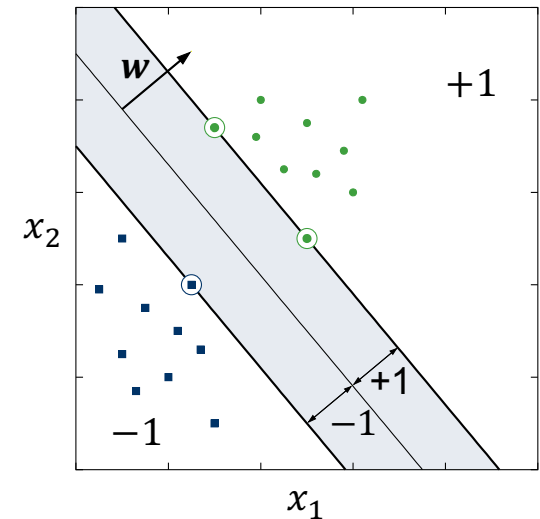
♦ Train the classifier by

- minimizing $\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \mathbf{w}^T \mathbf{w}$
- subject to $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 \geq 0$

i.e. data point should outside the margin
 y_i is class label (1/-1)

♦ Remarks

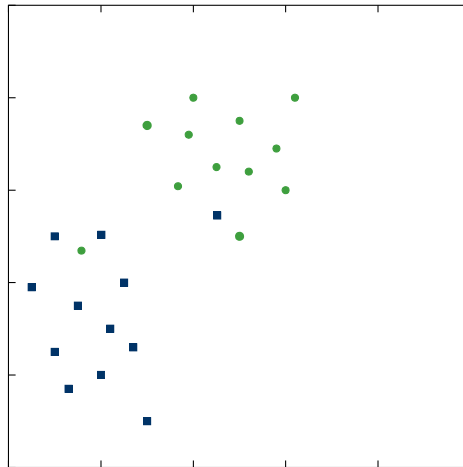
- this is a convex optimization problem
 - local optimum is always a global one – solution is unique
- there exist efficient algorithms for convex optimization
- standard libraries can be used



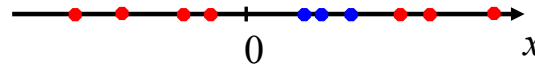
Great,
so we are done,
right?
No, not yet!



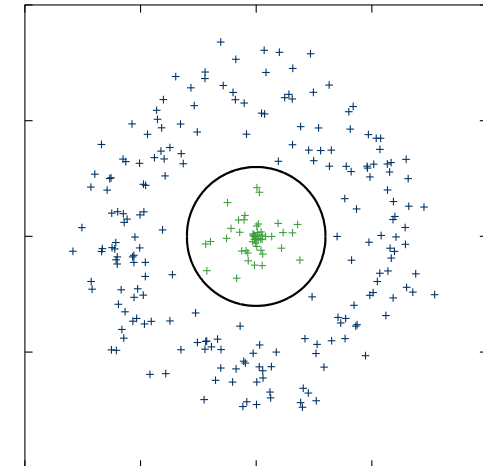
Non-linear Boundaries and Noisy Data



noise in the data



svm is one dimension lower and can only create one separation, not multiple



separation can only be linear

Three examples of not linearly separable classes

- ◆ Limitations of linear decision boundaries
 - non-linearly separable data cannot be classified
 - noisy data cause problems

➔ too simple for most practical purposes



Classification – Support Vector Machines (SVM)

1. Hard Margin SVM
2. Feature Transformations
3. The Kernel Trick
4. Regularization - Soft Margin SVM
5. SVM in practice



Feature Transformations

- ◆ Possible **Solution**

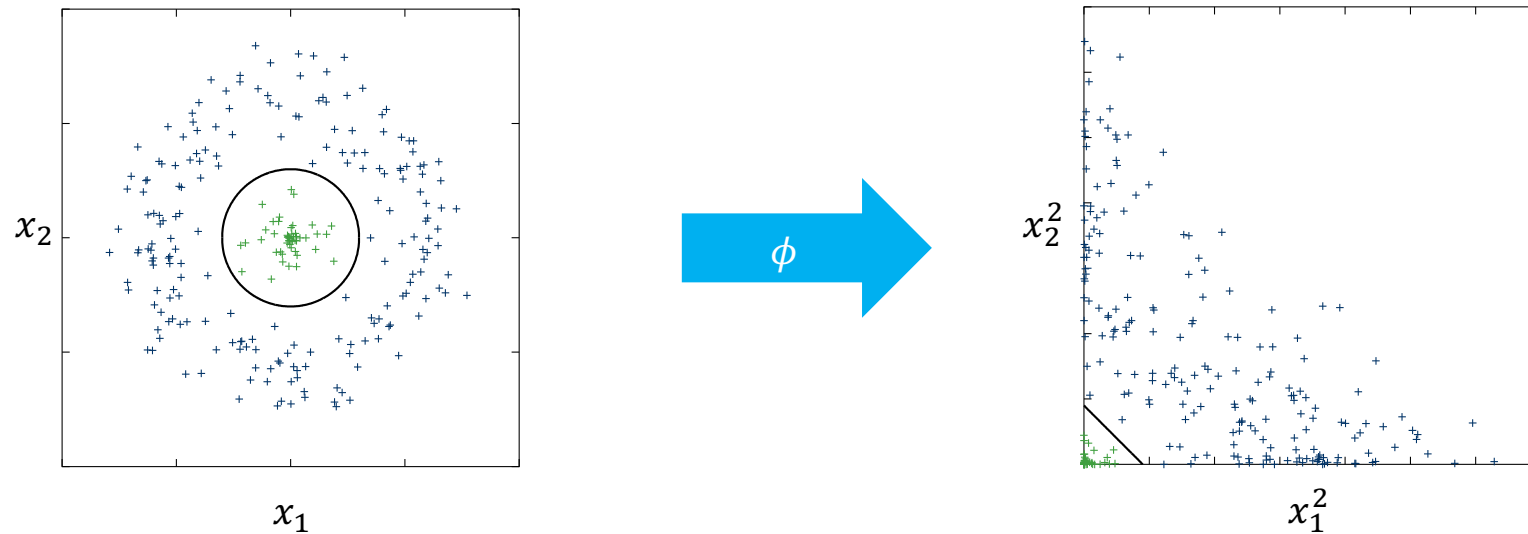
- Map data to higher dimensional feature space using non-linear feature transform,
- then use a linear classifier

- ◆ Mathematically:

Select a feature transform $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that the resulting features $\phi(x_i)$ are linearly separable.



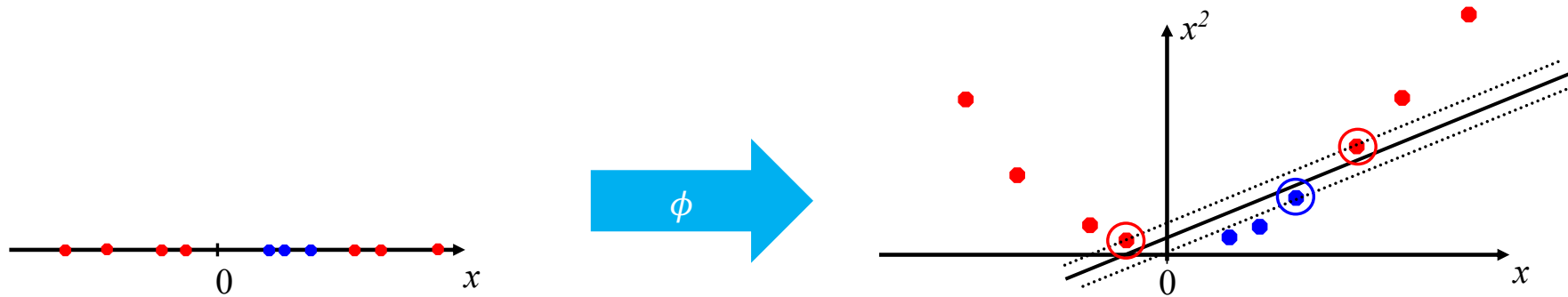
Feature Transformations – Example 1



- ♦ Applied feature transform in example: $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with $\phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ x_2^2 \end{pmatrix}$



Feature Transformations – Example 2



- ♦ Applied feature transform in example: $\phi: \mathbb{R}^1 \rightarrow \mathbb{R}^2$ with $\phi((x_1)) = \begin{pmatrix} x_1 \\ x_1^2 \end{pmatrix}$



Feature Transformations – Example 3

- ◆ Assume the decision boundary is defined by the quadratic function

$$f(\mathbf{x}) = a_0 + a_1x_1^2 + a_2x_2^2 + a_3x_1x_2 + a_4x_1 + a_5x_2$$

- ◆ This is obviously non-linear.
- ◆ Using the following mapping, we get features having a linear decision boundary:

$$\phi(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1^2 \\ x_2^2 \\ x_1x_2 \\ x_1 \\ x_2 \end{pmatrix} \quad \text{map each polynomial component}$$



Feature Transformations - in Generell

- ◆ Every dataset can be transformed into a linearly separable dataset with the right transformation
go one or more dimensions higher to make it separable (cf. ex 1, 2)
- ◆ New optimization problem
 - minimizing $\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \mathbf{w}^T \mathbf{w}$
 - subject to $y_i (\mathbf{w}^T \phi(\mathbf{x}_i) + w_0) - 1 \geq 0$
- ◆ The transformations may be very complex and/or be very high-dimensional
 - And we may need to try many transformations
- ➔ Explicit feature transformation very expensive (or even impossible)
- ➔ Optimization Problem very expensive

So now we
are done, right?
No, not yet!



Classification – Support Vector Machines (SVM)

1. Hard Margin SVM
2. Feature Transformations
3. The Kernel Trick
4. Regularization - Soft Margin SVM
5. SVM in practice



The Kernel Trick

Basic Idea

- ◆ **Step 1:** reformulate the original optimization problem using Lagrange-Multipliers
 - Note: we could do this including the feature transformation, but then the mathematical notations become harder to read, so we move this to a separate Step 3
- ◆ **Step 2:** convert to the Lagrangian-Dual-Problem
 - As our problem is a strongly dual problem (see literature for proof), the solutions are identical
- ◆ **Step 3:** apply the feature transformation
- ◆ **Step 4:** realize (and celebrate) that now we do not to explicitly compute the transformed vectors
only works on dual problems which is why we reformulate it to a lagrangian dual problem



Step 1: Optimization – Lagrangian

◆ Rewrite the constrained convex optimization problem in (primal) Lagrangian Form

- minimize $\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \mathbf{w}^T \mathbf{w}$
- subject to $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 \geq 0$
constraint would need to be applied for each data point, e.g. 10000 data points would lead to 10000 constraints

becomes

- minimize
$$L(\mathbf{w}, w_0, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_i \lambda_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1)$$

Lagrange multipliers

all data points (i amount)

constraints is added to optimization problem as a penalty instead and this equation is optimized instead
- subject to $\lambda_i \geq 0$
if $\geq$ then subtraction
if $\leq$ then addition



Step 2: Optimization – Lagrangian Dual (1)

- ◆ Reformulate as Lagrangian Dual L_D by Minimization of $\mathbf{w}$
 - Compute first partial derivatives
 - Set derivatives to zero
 - The resulting Lagrangian Dual L_D , is then maximized

$$L(\mathbf{w}, w_0, \lambda) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_i \lambda_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1)$$

$$\text{Partial derivative (1): } \frac{\partial L(\mathbf{w}, w_0, \lambda)}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \lambda_i y_i \mathbf{x}_i = 0 \quad \Rightarrow \mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$$

$$\text{Partial derivative (2): } \frac{\partial L(\mathbf{w}, w_0, \lambda)}{\partial w_0} = - \sum_i \lambda_i y_i = 0$$

the partial derivative for λ is not required for the following reformalization



Step 2: Optimization – Lagrangian Dual (2)

$$\begin{aligned} L_D &= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_i \lambda_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1) \\ &= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \underbrace{(\sum_i \lambda_i y_i \mathbf{x}_i)^T \mathbf{w}}_{= \mathbf{w}^T} - \underbrace{\sum_i \lambda_i y_i w_0}_{= 0} + \sum_i \lambda_i \\ &\quad \text{partial derivative (1)} \qquad \text{partial derivative (2)} \end{aligned}$$

$$= -\frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_i \lambda_i$$

$$= -\frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j + \sum_i \lambda_i \quad (\text{using partial derivative (1) again})$$

This term is now maximized subject to the constraints $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$

From previous slide:

$$\text{partial derivative (1): } \mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$$

$$\text{partial derivative (2): } \sum_i \lambda_i y_i = 0$$



Step 2: Resulting Optimization Problem

Training: Maximize $L_D = -\frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j + \sum_i \lambda_i$ subject to $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$

lambda is only non-zero for support vectors (which we are looking for)

Resulting hyperplane: $\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$ and $w_0 = y_k - \mathbf{w}^T \mathbf{x}_k$ for any $\mathbf{x}_k$ with non-zero λ_k

i.e. looking for lambda, y and x known from training data

Classification: $g(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x} + w_0) = \text{sgn}(\sum_i \lambda_i y_i \mathbf{x}_i^T \mathbf{x} + w_0)$

only x and w/wt are vectors, others are constants

- the $\mathbf{x}_i$ with non-zero λ_i are the support vectors
- the feature vectors $\mathbf{x}_i$ only appear in inner products
 - in training as well as classification phase
- $\mathbf{w}$ is not required explicitly for classification
 - we need only the support vectors and their Lagrange multipliers



Step 3: Feature transformation

- ◆ The feature transforms can be easily incorporated into SVMs

phi are the feature transforms this is the kernel trick and is replaced by any given kernel function that is needed and put into the equations below

- ◆ Replace $\mathbf{x}_i^T \mathbf{x}_j$ by $\phi^T(\mathbf{x}_i)\phi(\mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ ph transposed mult phi results in dot product

- Note: $\langle \cdot \rangle$ notation for inner product

for all vectors but actually only for support vectors

- ◆ Classification/Decision boundary: $g(\mathbf{x}) = \sum_i \lambda_i y_i \phi^T(\mathbf{x}_i)\phi(\mathbf{x}) + w_0 = \sum_i \lambda_i y_i \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle + w_0$

since lambda is 0 for non-support vectors

- ◆ Training: Lagrange dual problem

- Maximize $L_D = -\frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle + \sum_i \lambda_i$
 - subject to $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$

- ◆ the actual transform $\phi(\mathbf{x})$ is never required stand-alone: it only appears as inner products

➔ Kernel Trick



- ◆ In SVM training/classification, data appear only in the form of inner products $\langle \phi(x_i), \phi(x_j) \rangle$
- ◆ a Kernel-function is a function computing this inner product directly: $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$
 - i.e., without first transforming the features using $\phi(x)$
 - it can be computed in the original low-dimensional space!
- ◆ Kernel functions- can we use any $K(x_i, x_j)$ we like?
 - No, need to know we can convert it to the form $\langle \phi(x_i), \phi(x_j) \rangle$
 - Note: we do not need to know how to convert it to this form, only that it is possible!
 - Directly checking this is often cumbersome
 - Mercer's theorem: Any positive semi-definite symmetric function is a kernel
 - The “intermediate” feature space may even be ∞ -dimensional



Common Kernel Functions

- ◆ linear:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

- ◆ polynomial:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \langle \mathbf{x}_i, \mathbf{x}_j \rangle + r)^d \quad \text{d dimensional}$$

- ◆ Gaussian radial basis function (RBF):

$$K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2}} \quad \text{or} \quad K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2^2}$$

returns relationship in infinite dimension

- ◆ sigmoid:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \langle \mathbf{x}_i, \mathbf{x}_j \rangle + r)$$



Video

Video 1

The Polynomial Kernel

This is part 2 of a series on SVM by StatQuest

<https://www.youtube.com/watch?v=Toet3EiSFcM>





SVM – Classification with threshold

- ◆ Classification:

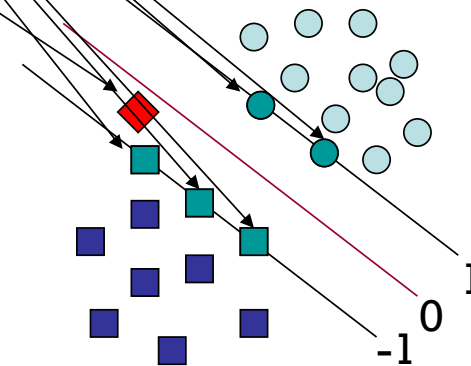
$$g(\mathbf{x}) = \sum_i \lambda_i y_i K(\mathbf{x}_i, \mathbf{x}) + w_0$$

- ◆ Classification without threshold

- decide for class based on $g(\mathbf{x}) < 0$ or $g(\mathbf{x}) > 0$

- ◆ Classification with confidence threshold t

- $g(\mathbf{x}) < -t$: class -1
- $g(\mathbf{x}) > t$: class $+1$
- $-t < g(\mathbf{x}) < t$: reject





The RBF-Kernel (Gaussian Kernel)

distance measure in infinite dimension

- ♦ The RBF-Kernel $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2^2}$
- ♦ maps to feature vectors into an ∞ -dimensional feature space
- ♦ Behaves like a weighted nearest-neighbor model
 - i.e., close data points in the training influence the classification more than far away ones
- ➔ it is always possible for a Gaussian kernel to perfectly separate any dataset
 - as long as there are no duplicate data points, and assuming exact arithmetic
- ➔ High variance / low bias technique
- ➔ Danger of Overfitting!



Video

Video 2

The Radial (RBF) Kernel

This is part 3 of a series on SVM by StatQuest

https://www.youtube.com/watch?v=Qc5IyLW_hns

gamma scales how much influence data points have on each other, the higher the gamma the more weak the influence



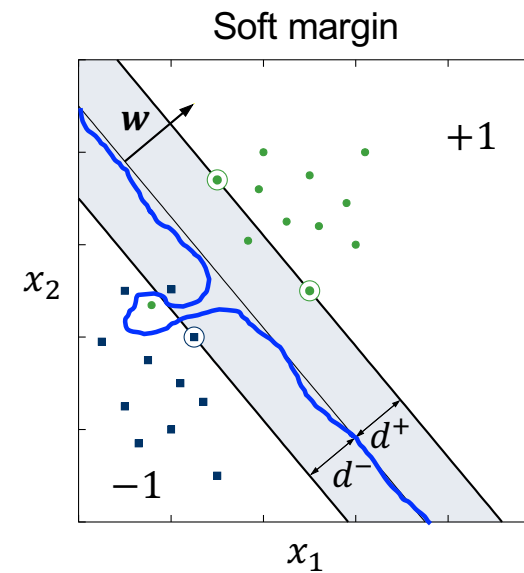
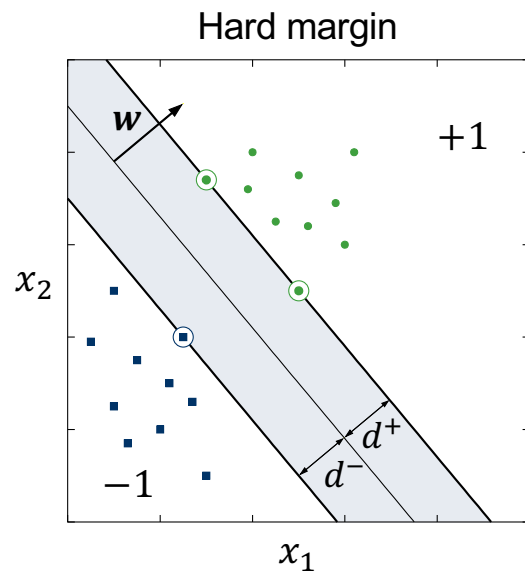


Classification – Support Vector Machines (SVM)

1. Hard Margin SVM
2. Feature Transformations
3. The Kernel Trick
4. Regularization - Soft Margin SVM
5. SVM in practice



Hard and Soft Margin Problem



plane in higher dimension
projecting it down would
result in picture to left

> overfitting
> not desirable

data are not linearly separable in this case



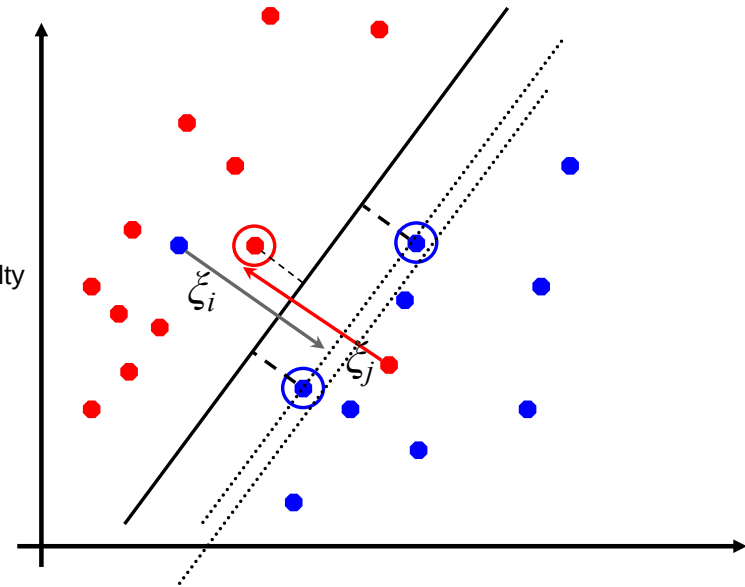
Soft Margin Regularization – Slack Variables (1)

Solutions: allow some errors

- allow misclassification of difficult or noisy samples
- move data slightly, to where they belong
- pay a penalty

Training

- minimize $\frac{1}{2} \|\mathbf{w}\|^2 + c \sum \xi_i$
penalty, the bigger c the higher the penalty
each point has to be on the correct side but is allowed error value
- subject to $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 + \xi_i \geq 0$
 $\xi_i \geq 0$





Soft Margin Regularization – Slack Variables (2)

◆ primal problem:

- minimize $\frac{1}{2} \|\mathbf{w}\|^2 + c \sum \xi_i$
- subject to $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 + \xi_i \geq 0$ and $\xi_i \geq 0$

◆ reformulate as dual problem: loss function does not change

- Maximize $L_D = -\frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle + \sum_i \lambda_i$
- subject to $0 \leq \lambda_i \leq c$ and $\sum_i \lambda_i y_i = 0$
 - neither the slack variables ξ_i
 - nor their Lagrange multipliers c appear in the target function

only change is additional constraint of $\lambda_i \leq c$

◆ c , trades off misclassification of training examples against simplicity of the decision surface.

- low c makes the decision surface smooth (more regularization = more bias = lower variance)
- high c aims at classifying all training examples correctly (higher variance = lower bias)



Classification – Support Vector Machines (SVM)

1. Hard Margin SVM
2. Feature Transformations
3. The Kernel Trick
4. Regularization - Soft Margin SVM
5. SVM in practice



Multiclass-SVM

- ◆ SVM so far: only binary classifications – how to extend to multi-class problems?

can only classify binary

➔ Split into multiple binary classifications

- ◆ one-vs-all

- one binary SVM per class, separating this class from all others, i.e., n classifiers
- winner-takes all strategy (winner = class with highest value)

- ◆ one-vs-one

- train binary SVMs for each pair of classes, i.e., $\frac{n(n-1)}{2}$ classifiers
- each SVM votes: max-wins strategy

- ◆ Usually, one-vs-one is preferable (but computationally more expensive)



SVM for classification in scikit-learn (as of v1.8.x)

- ◆ Expensive to train – may need (stratified) sampling
- ◆ Three different implementations for classification
 - **LinearSVC**: always uses linear Kernel
 - soft-margin with parameter C
 - one-vs-rest as default for multi-class
 - Implementation based on liblinear
 - **SVC**: very similar to what we have seen
 - incl. Kernel-Trick and soft-margin with parameter C
 - one-vs-one as default for multi-class
 - Implementation based on libsvm
 - **NuSVC**: very similar to SVC
 - Slightly different regularization parameter ν instead of C , which is easier to interpret
 - ν = upper bound on fraction of margin error and lower bound on fraction of support vectors
- ◆ see: <https://scikit-learn.org/stable/api/sklearn.svm.html>



Support Vector Regression

- ◆ Find $f(x) = \omega^T \phi(x) + b$ that is **as flat as possible** while keeping predictions within an **ϵ -tube**
 - points inside the tube: no loss
 - points outside: linear penalty (with slack)
 - Interpretation vs SVC: Instead of pushing points to be correctly classified with margin ≥ 1 , you're pushing residuals to be $\leq \epsilon$
- ◆ Margin-concept changes
 - SVC: margin = geometric distance between classes
 - SVR: margin = vertical tolerance band: the ϵ -tube
 - **Support vectors** are the samples that define the tube boundaries / violate them
- ◆ Hyperparameters
 - C still trades off flatness / regularization vs violations (like “soft margin”)
 - ϵ sets the tube width
 - larger ϵ : fewer support vectors, smoother fit, more bias
 - smaller ϵ : tighter fit, more support vectors, more variance / sensitivity



SVR for classification in scikit-learn (as of v1.8.x)

- ◆ Expensive to train – may need sampling
- ◆ Three different implementations for classification
 - **LinearSVR**: always uses linear Kernel
 - soft-margin with parameters C and ϵ
 - Implementation based on liblinear
 - **SVR**: very similar to what we have seen
 - incl. Kernel-Trick and soft-margin with parameters C and ϵ
 - Implementation based on libsvm
 - **NuSVR**: very similar to SVR
 - Slightly different regularization parameter ν instead of ϵ (!), which is easier to interpret
 - C = penalty of the error term
 - ν = upper bound on fraction of training error and lower bound on fraction of support vectors
- ◆ see: <https://scikit-learn.org/stable/api/sklearn.svm.html>



References

slides based on

- ♦ slides of the lecture *Pattern Recognition* taught at the FAU Erlangen-Nuremberg, courtesy of D. Hahn, J. Hornegger, S. Steidl and E. Nöth.
- ♦ Ray Mooney: Support Vector Machines. Slides, University of Texas at Austin.
- ♦ Ch. Manning, P. Nayak: Introduction to Information Retrieval, Lecture 14: Support vector machines and machine learning documents. Stanford University.



Key Takeaways

- ◆ Support Vector Machines – basic idea
- ◆ Kernel Trick
- ◆ Regularization
- ◆ Kernel und Hyperparameters (C , ν) need to be chosen carefully / tuned extensively

the higher the kernel the higher the similarity between two points

the smaller the smaller the similarity between two points

kernel is not distance between two points and instead describes relationship between two points (see above)





Exercises

Exercise 1

SVM in scikit_learn

